

Metodi Informatici per la Gestione Aziendale

Sistema di Raccomandazione e Sentiment Analysis

Amazon Reviews 2023, categoria Movies_and_TV

Progetto d'esame, Livello AVANZATO

Andrea Saverino, A.A. 2025/2026

Indice

Indice.....	2
Executive summary.....	3
1. Introduzione e descrizione dei dati.....	4
1.1 Obiettivo e impostazione del lavoro	4
1.2 Il dataset e le sue colonne	4
1.3 Campionamento e pulizia iniziale	5
1.4 Ambiente di lavoro e riproducibilità.....	6
2. Analisi esplorativa (EDA).....	7
3. Preparazione dei dati: k-core e dimensionamento.....	10
3.1 Il filtro k-core.....	10
3.2 I limiti dimensionali (e come li ho scelti)	10
4. Parte base, Collaborative Filtering.....	11
4.1 K-NN: elbow e grid search	11
4.2 SVD (matrix factorization).....	12
4.3 Confronto dei modelli.....	12
4.4 Le liste Top-N	13
4.5 Segmentazione degli utenti con K-Means	14
5. Parte intermedia, Content-Based Filtering.....	16
5.1 Metadati e preprocessing.....	16
5.2 I due embedding: TF-IDF e DistilBERT	16
5.3 Predizione dei rating e confronto tra embedding.....	17
5.4 Content-based contro collaborative filtering.....	18
6. Parte avanzata, Sentiment Analysis	19
6.1 Etichette, sbilanciamento e campione	19
6.2 Le feature: TF-IDF e transformer.....	20
6.3 I classificatori e la gestione dello sbilanciamento	20
7. Bonus, Sentiment con un LLM (Groq)	23
8. Analisi aggiuntive oltre il programma (versione estesa)	25
8.1 KNNBaseline: il K-NN con correzione dei bias.....	25
8.2 SVD sul k-core completo: mettere alla prova i tetti	25
8.3 SBERT: un embedding specializzato nella similarità.....	27
8.4 Classificatori lineari: LinearSVC e Logistic Regression	28
8.5 Il quadro complessivo dei test	29
9. Limiti e scelte metodologiche.....	29
10. Conclusioni.....	30

Executive summary

In questo progetto ho costruito, in tre fasi incremental, un sistema di raccomandazione e di analisi del sentiment sulla categoria Movies_and_TV di Amazon Reviews 2023 (circa 17 milioni di recensioni, di cui ho usato un campione casuale di un milione). L'idea di fondo è partire dal segnale più semplice, cioè chi ha dato quale voto a quale film, e arricchirlo passo dopo passo: prima un collaborative filtering basato solo sui voti, poi un content-based che sfrutta i testi descrittivi dei film, infine una sentiment analysis sulle recensioni scritte, chiusa da un confronto con un LLM. Gli strumenti sono quelli dei laboratori del corso: Surprise, scikit-learn, NLTK, Hugging Face e langchain-groq. Le analisi aggiuntive condotte con strumenti oltre il programma sono raccolte nel capitolo 8.

Nella fase base il modello migliore è risultato l'SVD con RMSE 0,9401: quando prevede un voto, in media sbaglia di poco meno di una stella su una scala da 1 a 5. Il K-NN si ferma invece a 1,0630, quindi l'SVD guadagna l'11,6%, ottimizzando entrambi con grid search in cross-validation a 5 fold. La segmentazione con K-Means, scegliendo il numero di cluster con Elbow e Silhouette, ha trovato due gruppi molto ben separati (silhouette 0,785) e facili da interpretare: utenti “critici” (voto medio 3,69) e utenti “generosi” (4,67).

Nella fase intermedia i due embedding testuali si sono rivelati di fatto equivalenti (TF-IDF: RMSE 0,9836; DistilBERT: 0,9835 su 28.373 coppie di test), a patto di dare al transformer il testo grezzo e non quello pre-processato. Il confronto con il collaborative filtering, condotto sullo stesso split e sulle stesse coppie di test, conferma la superiorità del CF (SVD: 0,9498), lasciando al content-based il ruolo insostituibile di risposta al cold-start dei prodotti nuovi.

Nella fase avanzata il miglior classificatore del corso è il Random Forest su TF-IDF (F1 macro 0,621), con la classe “neutro” confermata come la più difficile. Il risultato più interessante arriva dal bonus: l'LLM interrogato via Groq, senza alcun addestramento, batte il classificatore addestrato sulle stesse 300 recensioni (F1 0,754 contro 0,572). Le analisi aggiuntive, infine, mostrano che due estensioni migliorano i risultati (LinearSVC: F1 0,708; KNNBaseline: RMSE 1,0021) e due esperimenti confermano le scelte fatte (l'SVD senza limite di utenti peggiora; SBERT non supera gli altri embedding).

1. Introduzione e descrizione dei dati

1.1 Obiettivo e impostazione del lavoro

Un sistema di raccomandazione è un programma che prova a indovinare che cosa piacerà a un utente, per suggerirglielo: è quello che fanno Netflix o Amazon quando propongono un titolo. L'obiettivo del progetto è costruirne uno completo sulla categoria dei film, e affiancargli un'analisi del sentiment delle recensioni scritte.

L'impostazione è incrementale, perché le tre fasi usano informazioni sempre più ricche. La parte base usa solo la matrice dei voti utenti-film: è il segnale essenziale e fa da termine di paragone per il resto. La parte intermedia aggiunge i metadati testuali dei film (titolo e descrizione), per capire se “ciò di cui parla” un film basta a prevedere quanto piacerà. La parte avanzata lavora invece sul testo delle recensioni e si chiede se dalle parole si possa ricavare il giudizio. Come strumenti ho scelto di restare sui metodi e sulle librerie dei laboratori: Lab 8 per i recommender system (Surprise), Lab 9 per l'NLP a frequenza, Lab 10 per i transformer, Lab 6 per la classificazione, Lab 7 per il clustering e Lab 11 per l'approccio LLM.

Una precisazione sul modo in cui misuro i risultati, visto che tutto il report ci ritorna sopra. Prevedere il voto che un utente darà a un film è un problema di regressione, e la metrica di riferimento è l'RMSE, la radice dell'errore quadratico medio: per ogni coppia utente-film del test si calcola la differenza tra voto previsto e voto reale, la si eleva al quadrato, se ne fa la media e se ne prende la radice. Il risultato è espresso nella stessa unità dei voti, cioè in stelle, e si legge così: un RMSE di 1,0 vuol dire che il modello sbaglia mediamente di una stella su cinque. Riporto sempre anche l'MSE, che è lo stesso numero prima della radice: ordina i modelli allo stesso modo, ma è meno leggibile perché non è più in stelle. L'elevamento al quadrato ha una conseguenza che vale la pena tenere a mente, cioè che gli errori grandi pesano molto più di quelli piccoli: un modello che sbaglia di tre stelle su poche previsioni viene penalizzato più di uno che sbaglia di mezza stella su tutte. Nella parte avanzata il problema diventa di classificazione e la metrica cambia: ne parlo nel capitolo 6.

1.2 Il dataset e le sue colonne

Il dataset è Amazon Reviews 2023 del gruppo McAuley-Lab, disponibile su HuggingFace e diviso per categorie. È composto da due file. Il primo contiene le recensioni degli utenti, con le colonne che ho effettivamente usato: `user_id` (l'identificativo dell'utente), `parent_asin` e `asin` (gli identificativi del prodotto, uso `parent_asin` perché raggruppa le varianti dello stesso titolo, ad esempio DVD e Blu-ray, in un unico film), `rating` (il voto da 1 a 5 stelle), `title` e `text` (titolo e corpo della recensione scritta, usati nella parte di sentiment), `timestamp` (data della recensione, che uso per la deduplicazione), più `helpful_vote` e `verified_purchase` per l'analisi esplorativa. Il secondo file contiene i metadati dei prodotti, da cui prendo `title` e `description`: sono il “contenuto” su cui lavora la parte intermedia.

Il file delle recensioni ha in tutto dieci campi. Oltre a quelli elencati contiene `images`, cioè le fotografie che gli utenti caricano dopo aver ricevuto il prodotto, disponibili in tre risoluzioni. Le ho scartate già in fase di caricamento: non servono a nessuna delle tre fasi e sono di gran lunga la voce che pesa di più

sulla memoria. Il file dei metadati ne ha tredici: oltre a title e description, che sono il materiale della parte intermedia, contiene main_category, average_rating e rating_number (il voto medio e il numero di voti mostrati sulla pagina del prodotto), features, price, images, videos, store, categories, details e bought_together. Di questi ho tenuto soltanto i due campi testuali indicati dalla traccia, sia per coerenza con la consegna sia per una ragione sostanziale: titolo e descrizione sono gli unici attributi disponibili anche per un titolo appena pubblicato, che è esattamente il caso in cui un sistema content-based serve davvero.

La scelta di parent_asin come identificativo di prodotto merita una riga in più, perché determina la matrice su cui lavora tutto il progetto. Su Amazon lo stesso film esiste in molte varianti, DVD, Blu-ray, edizione speciale, cofanetto e ognuna ha un asin diverso. Usando asin, i voti di uno stesso titolo si spargerebbero su prodotti distinti, ciascuno con pochissime recensioni: proprio la frammentazione che il collaborative filtering non tollera. Il campo parent_asin raggruppa le varianti sotto un unico identificativo e concentra il segnale dove serve. Nel caricamento uso quindi parent_asin, ricadendo su asin solo nei rari casi in cui il primo è mancante.

Sul piano dimensionale, Movies_and_TV è una delle categorie più grandi del dataset: 17,3 milioni di voti scritti da 6,5 milioni di utenti su 747,8 mila titoli, per circa un miliardo di token di testo nelle recensioni e 415,5 milioni nei metadati. È una taglia che rende il campionamento inevitabile, ma che garantisce anche che un campione da un milione di voti non sia un caso limite: resta abbastanza grande da conservare tutte le caratteristiche strutturali della categoria.

Tra le categorie disponibili ho scelto Movies_and_TV per tre ragioni. Le recensioni di film sono lunghe, argomentate e polarizzate: materiale ideale per la sentiment analysis. I metadati hanno titolo e descrizione quasi sempre compilati, il che dà basi solide al content-based. E il rapporto tra voti e utenti è buono, quindi dopo il filtraggio la matrice resta abbastanza densa per il collaborative filtering.

1.3 Campionamento e pulizia iniziale

La categoria conta circa 17 milioni di recensioni: troppe per la memoria di Colab. Ho quindi letto il file in streaming (una riga alla volta, senza caricarlo tutto), mescolando il flusso con un buffer di 100.000 righe e seed fisso 42, e mi sono fermato a 1.000.000 di recensioni. In questo modo il campione così ottenuto è davvero casuale, non le “prime righe del file” che potrebbero essere ordinate per data o prodotto, ed è riproducibile grazie al seed.

Vale la pena esplicitare che cosa questo campionamento sia e che cosa non sia. La funzione di streaming legge il file una riga alla volta e non lo carica mai per intero, ma proprio per questo non può estrarre un campione uniforme sull'intero file: mantiene un buffer di 100.000 righe dal quale pesca a caso, rimpiazzando via via le righe consumate. Non è quindi un campionamento casuale semplice sui 17,3 milioni di record, ma è enormemente più vicino a esserlo del semplice troncamento alle prime righe, che nel file originale sono ordinate e avrebbero prodotto un campione concentrato su pochi prodotti e su un intervallo temporale ristretto. È il compromesso che mi ha permesso di lavorare su un milione di voti restando dentro i limiti di memoria dell'ambiente.

In fase di caricamento tengo solo le colonne che servono alla parte base: `user_id`, `parent_asin`, `asin`, `rating`, `timestamp`, `helpful_vote` e `verified_purchase`, scartando testo e immagini. Il testo delle recensioni non entra nel collaborative filtering, che lavora sui soli voti, e viene ricaricato all'inizio della parte avanzata, dove serve davvero: tenere separati i due caricamenti evita di trascinarsi in memoria un milione di recensioni testuali mentre si calcolano matrici di similarità.

Sulla base grezza ho fatto subito una pulizia: se lo stesso utente ha recensito due volte lo stesso film, tengo solo il giudizio più recente. I duplicati trovati sono 2.594 (lo 0,26% del campione): pochi, ma senza questa pulizia una stessa coppia utente-film peserebbe due volte nel collaborative filtering. Restano così 997.406 recensioni.

Il percorso dei dati, dal file grezzo alla matrice su cui girano i modelli, attraversa in tutto quattro stadi: il milione di recensioni estratte in streaming, le 997.406 che restano dopo la deduplicazione, le 463.509 che sopravvivono al filtro k-core del capitolo 3 e le 141.863 del dataset finale, ristretto ai 4.000 utenti più attivi e ai 12.500 titoli più recensiti per rientrare nella memoria disponibile. Ogni passaggio è motivato nel punto in cui viene applicato; li elenco insieme qui perché i numeri che compaiono nel resto del report si riferiscono a stadi diversi a seconda del contesto, e conviene avere chiaro fin dall'inizio a quale.

1.4 Ambiente di lavoro e riproducibilità

Ho sviluppato ed eseguito tutto su Google Colab con GPU (necessaria per gli embedding neurali). Il seed è fissato a 42 in ogni componente; ogni figura e tabella viene salvata automaticamente in `report/figure/` e gli output dei modelli in `report/risultati/`, così ogni numero di questo report è ricostruibile dal notebook.

Sull'ambiente due dettagli meritano una nota. Il primo riguarda Surprise, che va compilata contro la versione di NumPy presente nel runtime: il notebook la installa senza isolamento della build (`--no-build-isolation`), così che la compilazione usi esattamente la versione già caricata dal runtime: la compatibilità binaria è garantita e l'intero notebook gira dall'inizio alla fine senza alcun riavvio del kernel. La sessione da cui provengono i risultati di questo report girava su Python 3.13.15 con NumPy 2.1.3, scikit-learn 1.6.1 e scikit-surprise 1.1.5. Il secondo è che tutti i parametri del progetto, come: categoria, dimensione del campione, soglia k-core, numero di fold, lunghezza delle liste di raccomandazione, tetti dimensionali, modelli usati per gli embedding e per l'LLM, si trovano in un'unica cella di configurazione all'inizio del notebook: rieseguire l'intera analisi su un'altra categoria merceologica richiede di cambiare una riga sola.

2. Analisi esplorativa (EDA)

Prima di costruire i modelli ho guardato i dati per rispondere a quattro domande: quanto è densa la matrice dei voti (la più importante, perché da lì dipende la fattibilità del collaborative filtering), come sono distribuiti i voti sulla scala da 1 a 5, come si distribuiscono le recensioni fra utenti e prodotti, e se qualcuna delle variabili numeriche disponibili spieghi il voto da sola. Le prime tre condizionano direttamente la parte base; l'ultima serve a verificare se esista una scorciatoia, cioè prevedere il rating con un modello semplice sulle sole colonne numeriche, prima di costruire i sistemi di raccomandazione veri e propri.

Il campione conta 997.406 recensioni scritte da 185.054 utenti su 211.512 prodotti. Messi in matrice, questi numeri danno una densità dello 0,0025%: in media ogni utente ha votato 5,4 film e ogni film ha ricevuto 4,7 voti. Detto in un altro modo: su 40.000 celle della matrice, una sola contiene un voto. Una matrice così vuota non permette a un algoritmo collaborativo di trovare abbastanza confronti, e giustifica il filtro del capitolo 3.

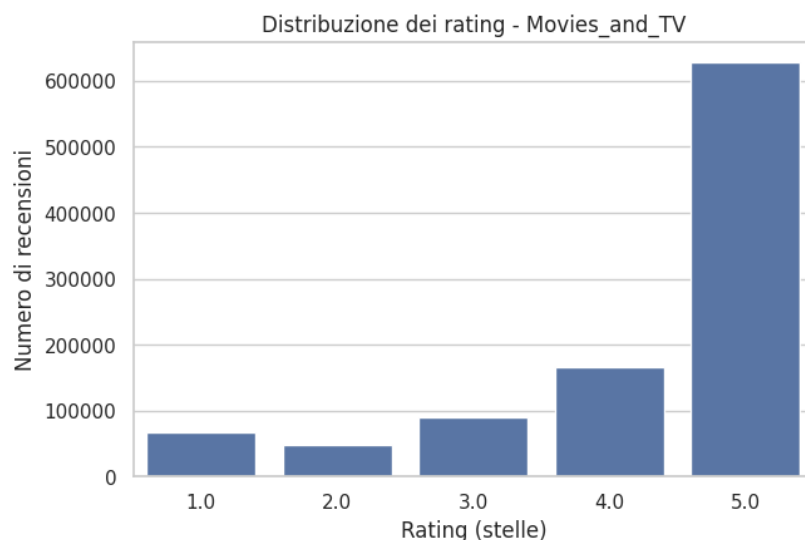


Figura 2.1. Distribuzione dei rating: le 5 stelle dominano.

La distribuzione dei voti dice una cosa nota a chiunque legga recensioni online: chi scrive lo fa soprattutto quando è soddisfatto. I voti positivi (4-5 stelle) coprono quasi l'80% del campione. È un dato da tenere presente, perché questo sbilanciamento tornerà due volte: nella sentiment analysis, dove diventa il problema centrale, e nel clustering, dove separerà gli utenti per stile di voto.

In numeri, raggruppando le stelle nelle tre fasce che userò nella parte avanzata: 795.715 recensioni da 4-5 stelle (79,6%), 89.339 da 3 stelle (8,9%) e 114.946 da 1-2 stelle (11,5%). I conteggi sono calcolati sul milione di recensioni grezze, prima di rimuovere i duplicati; poiché questi ultimi incidono solo per lo 0,26%, le proporzioni sulle 997.406 recensioni pulite restano di fatto invariate, e il notebook lo verifica riportando anche quei conteggi: 793.571, 89.144 e 114.691, cioè le stesse quote percentuali. Il dato più interessante non è la prevalenza dei voti alti, che è attesa, ma il fatto che la fascia intermedia sia la più rara delle tre: chi ha un'opinione tiepida tende semplicemente a non scrivere. È questa la ragione per cui, nel capitolo 6, la classe neutro risulterà la più difficile da riconoscere per ogni classificatore che ho provato.

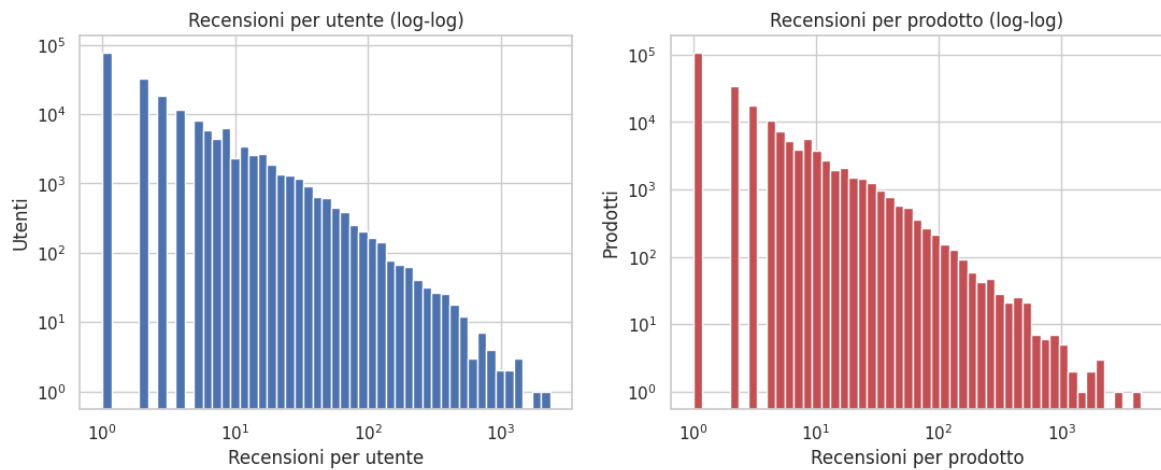


Figura 2.2. Coda lunga (scala log-log): recensioni per utente e per prodotto.

I due grafici in scala logaritmica mostrano la classica “coda lunga”: il 41,6% degli utenti ha scritto una sola recensione e il 51,1% dei prodotti ne ha ricevuta una sola, mentre pochi utenti molto attivi e pochi titoli molto popolari concentrano gran parte delle interazioni. È un secondo argomento a favore del filtro: la maggior parte di utenti e prodotti, da sola, non porta segnale utilizzabile.

La coda lunga ha una conseguenza diversa su ciascuna delle tre parti del progetto. Sul collaborative filtering pesa moltissimo: un utente con una sola recensione non ha alcun film in comune con nessun altro e non è confrontabile, ed è il motivo per cui serve il filtro k-core applicato nella fase successiva. Sul content-based pesa in modo opposto: un titolo con una sola recensione è quasi un caso di cold-start, e sono proprio questi i prodotti su cui la descrizione testuale è l'unica informazione disponibile, cioè quelli su cui questo approccio dà il suo contributo. Sulla sentiment analysis, infine, non pesa affatto: lì ogni recensione vale come esempio a sé, indipendentemente da quante ne abbia scritte il suo autore, ed è per questo che la parte avanzata può lavorare sul campione grezzo senza applicare alcun filtro.

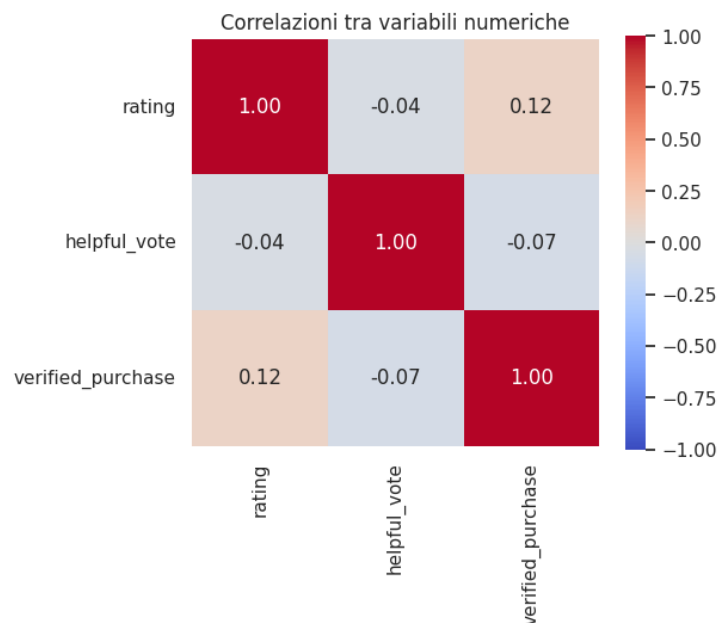


Figura 2.3. Correlazioni tra le variabili numeriche.

La matrice di correlazione copre le tre variabili numeriche del file delle recensioni: il voto, i voti utili ricevuti dalla recensione e l'acquisto verificato codificato come 0/1. Il legame tra voto e acquisto verificato vale 0,12, quello tra voto e voti utili $-0,04$ e quello tra voti utili e acquisto verificato $-0,07$. Sono tutti coefficienti piccoli, nell'ordine di grandezza in cui una correlazione non ha alcun valore predittivo; il più forte dei tre dice soltanto che chi ha comprato davvero il prodotto tende a votarlo un po' più alto. Il segno negativo fra voto e voti utili è coerente con l'intuizione che una critica argomentata aiuti a decidere più di un elogio generico: le recensioni giudicate utili tendono, di pochissimo, a essere quelle meno entusiaste. Il coefficiente resta però troppo piccolo per ricavarne qualcosa di più di un'osservazione. Va poi detto che `helpful_vote` è una variabile fortemente asimmetrica: mediana e terzo quartile valgono entrambi zero, solo il 24,7% delle recensioni riceve almeno un voto utile, la media è 1,14 e il massimo osservato 2.162. Una correlazione lineare su una distribuzione così sbilanciata va letta come indicazione debole, non come una quantificazione affidabile del legame fra le due grandezze. Il voto, insomma, non è spiegabile da nessuna delle variabili di contorno presa a sé: la scorciatoia non esiste, e per prevederlo servono davvero i modelli delle fasi successive. È un esito atteso, ed è proprio quello che giustifica l'impostazione dell'intero lavoro.

Vale infine la pena dire che cosa l'analisi esplorativa non può dirmi. Il dataset non contiene informazioni sugli utenti al di là del loro identificativo: né età, né provenienza, né storia d'acquisto fuori dalla categoria. Ogni caratterizzazione di un utente, in questo progetto, può quindi nascere soltanto dai voti che ha dato o dai testi che ha scritto. È il motivo per cui la segmentazione degli utenti lavora sul profilo di voto ricostruito e non su attributi anagrafici, e per cui il cold-start affrontato nel content-based riguarda i prodotti nuovi ma non gli utenti nuovi, per i quali questo dataset non offre alcuna informazione alternativa ai voti che ancora non esistono.

3. Preparazione dei dati: k-core e dimensionamento

3.1 Il filtro k-core

Il collaborative filtering ha bisogno di sovrapposizione: per dire che due utenti si somigliano, devono aver votato dei film in comune. Con utenti da una recensione la sovrapposizione non esiste. Il rimedio standard è il filtro k-core: tengo solo utenti e film con almeno k voti. Il filtro va ripetuto iterativamente, perché togliendo gli utenti poco attivi alcuni film scendono sotto soglia e viceversa; l'iterazione continua finché la matrice non è stabile. La soglia non l'ho fissata a caso: ho confrontato quattro valori.

k	Utenti	Prodotti	Rating	% rating	Densità
3	66.092	61.435	669.857	67,2%	0,017%
5	33.698	30.923	463.509	46,5%	0,045%
10	7.326	6.773	146.795	14,7%	0,296%
20	0	0	0	0%	0%

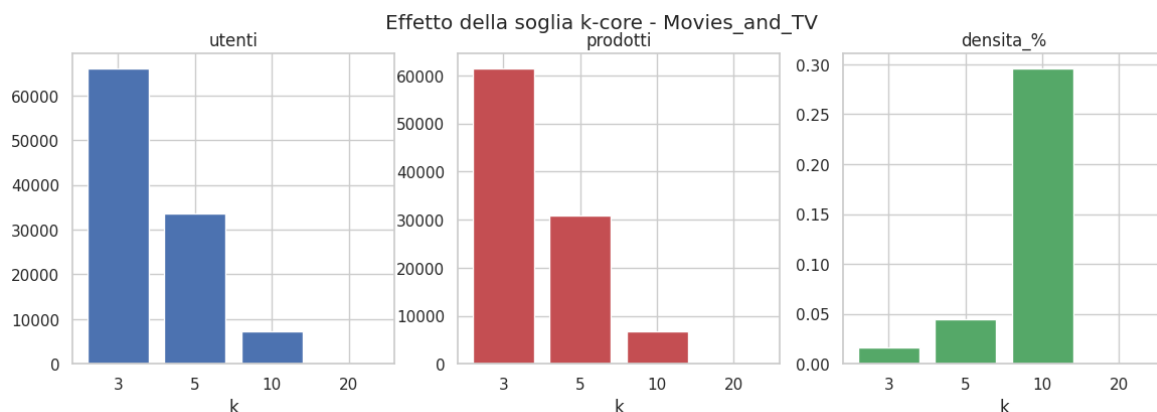


Figura 3.1. Effetto della soglia k-core su utenti, prodotti e densità.

La tabella è un compromesso tra quantità e qualità dei dati. Con k=3 tengo due terzi dei voti ma la matrice resta troppo vuota. Con k=10 la densità decolla ma butto via l'85% dei dati. Con k=20 succede una cosa istruttiva: il dataset si svuota del tutto, nel campione non esiste un nucleo di utenti e film che si votano a vicenda con quell'intensità. Ho scelto k=5, il punto di equilibrio: 463.509 voti, 33.698 utenti, 30.923 film, densità decuplicata.

3.2 I limiti dimensionali (e come li ho scelti)

Dopo il k-core ho applicato due tetti, entrambi per lo stesso motivo tecnico: il K-NN di Surprise costruisce matrici di similarità che crescono con il quadrato del numero di utenti (versione user-based) e del numero di prodotti (versione item-based). Il primo tetto tiene i 4.000 utenti più attivi. Il secondo l'ho aggiunto dopo averne verificato la necessità in fase di esecuzione: con il campione da un milione i prodotti sopravvissuti al filtro erano 14.594, e la matrice item-item da 14.594^2 celle ha saturato la RAM di Colab facendo cadere la sessione. Ho quindi fissato un tetto di 12.500 prodotti (i più recensiti), dimensionato empiricamente per stare dentro la memoria disponibile. Dopo ogni tetto riapplico il k-core, così la matrice resta coerente.

Il dataset finale della parte base è: 141.863 voti, 4.000 utenti, 12.500 film, densità 0,284%. Questa scelta orienta l'analisi verso gli utenti più attivi; per questo non l'ho lasciata come assunzione, ma l'ho messa alla prova con un esperimento dedicato (capitolo 8.2). L'esito è stato sorprendente: il dataset con i tetti non è solo più leggero, è anche quello su cui il modello migliore lavora meglio.

4. Parte base, Collaborative Filtering

Il collaborative filtering parte da un'idea semplice: se io e un altro utente abbiamo dato voti simili agli stessi film, probabilmente mi piaceranno anche i film che sono piaciuti a lui e che io non ho ancora visto. Non serve sapere nulla dei film, né genere né trama: bastano i voti. In questa parte confronto due algoritmi: il K-NN, che applica l'idea alla lettera cercando i “vicini” più simili, e la matrix factorization SVD, che la applica in modo più astratto. Il vincitore riempie la matrice, genera le raccomandazioni e alimenta la segmentazione.

4.1 K-NN: elbow e grid search

Per l'algoritmo K-NN bisogna domandarsi due cose: quanti vicini considerare e con quale misura di similarità confrontarli. Ho affrontato le due domande separatamente. Come primo passo ho esplorato il solo numero di vicini k con l'elbow method su una configurazione di partenza (KNNBasic con parametri di default): allenandolo con k crescenti si vede dove l'errore smette di scendere. La curva che ne esce è però quasi piatta: fra $k=10$ e $k=50$ l'RMSE oscilla tra 1,1665 e 1,1686, cioè dentro un intervallo di due millesimi, e il minimo cade a $k=20$ senza che si veda un gomito riconoscibile. È già di per sé un'informazione utile: con la configurazione di partenza il numero di vicini, da solo, non sposta l'errore, e la leva vera va cercata altrove. Per questo non ho fissato k da questa curva, ma l'ho messo in griglia insieme agli altri due iperparametri. Come secondo passo ho quindi eseguito una grid search completa in cross-validation a 5 fold, combinando le tre misure di similarità del corso (msd, cosine, pearson) con l'approccio user-based e item-based e i valori di k in {20, 30, 40}.

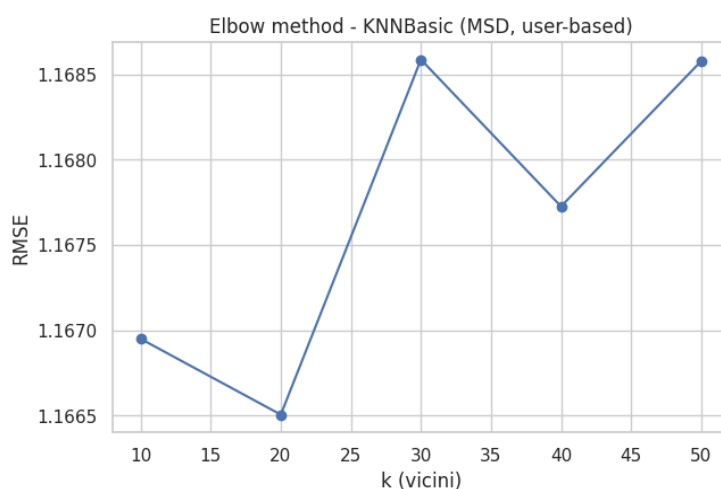


Figura 4.1. Elbow method sul numero di vicini k .

Dalla grid search è emersa una combinazione migliore di quella usata nell'elbow: item-based con similarità cosine e $k=40$, che porta l'RMSE a 1,0630 (contro il ~1,167 della configurazione di default vista in figura). Un errore medio di 1,06 nella scala dei voti significa che, quando il modello prevede un voto,

in media sbaglia di circa una stella su cinque. Il fatto che a vincere sia l'approccio item-based ha una spiegazione intuitiva: i profili dei film (chi li ha votati e con che voto) sono più ricchi e stabili di quelli degli utenti, e questo rende le somiglianze tra film più affidabili delle somiglianze tra persone.

4.2 SVD (matrix factorization)

L'SVD affronta il problema da un'altra strada. Invece di cercare vicini, suppone che dietro ai gusti ci siano dei fattori nascosti (per esempio quanto un film è d'azione, quanto è romantico, quanto è d'autore) e li scopre da solo dai numeri: ogni film viene descritto da quanto possiede ciascun fattore, ogni utente da quanto lo ama, e il voto previsto è quanto i due profili combaciano. Anche qui ho fatto una grid search in cross-validation a 5 fold, testando due iperparametri: il numero di fattori latenti (provato a 50 e a 100) e il termine di regolarizzazione (provato a 0,02 e a 0,05). Ha vinto la combinazione più semplice, 50 fattori con regolarizzazione 0,05, portando l'RMSE a 0,9401. Che la variante da 100 fattori faccia peggio non sorprende: con 4.000 utenti raddoppiare i parametri produce overfitting, ossia il modello si adatta anche alle variazioni casuali dei voti di training e generalizza peggio sui voti che non ha mai visto.

4.3 Confronto dei modelli

Modello	RMSE	MSE	Configurazione
SVD	0,9401	0,8838	50 fattori, reg 0,05
K-NN (KNNBasic)	1,0630	1,1299	cosine, item-based, k=40

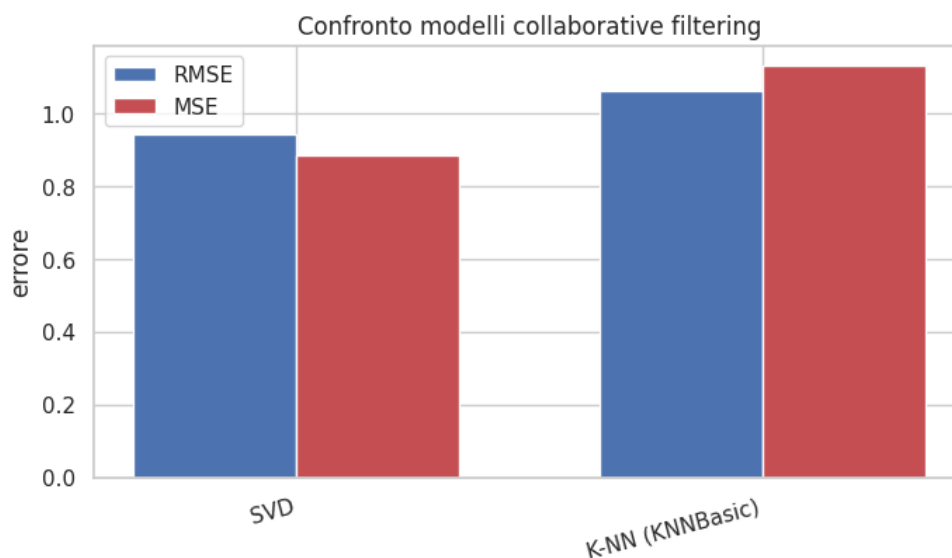


Figura 4.2. Confronto RMSE/MSE tra K-NN e SVD (5-fold CV).

L'SVD riduce l'errore dell'11,6%. La ragione è strutturale. Il K-NN può basarsi solo sui film votati in comune, che su una matrice sparsa sono pochi: se due utenti simili non hanno titoli in comune, per il K-NN non sono simili affatto. L'SVD invece impara una rappresentazione compressa che generalizza, perché due utenti con gusti affini finiscono vicini nello spazio dei fattori latenti anche senza aver votato gli stessi film. Ho quindi riallenato l'SVD vincente su tutti i dati e l'ho usato per il filling, cioè il completamento della matrice con i voti previsti per ogni coppia utente-film mancante.

Il filling l'ho eseguito con entrambi i modelli, e non solo con il vincitore, perché il livello avanzato lo prevede in due passi distinti: prima il completamento della matrice con la configurazione ottimale del K-NN, poi lo stesso completamento tramite matrix factorization, e infine il confronto fra i due. I due riempimenti agiscono sulla stessa matrice 4.000 utenti per 12.500 film, cinquanta milioni di celle di cui solo 141.863 osservate e si differenziano unicamente per il modello che genera i valori mancanti. Materializzare cinquanta milioni di celle due volte non sta nella memoria di Colab: su file salvo quindi il riempimento di un campione casuale di 200 utenti, lo stesso campione per entrambi i modelli, così che le due matrici siano confrontabili cella per cella; le metriche su cui poggia il confronto restano invece quelle validate in cross-validation sull'intero dataset. Le due matrici stimate hanno le stesse medie (4,29 il K-NN, 4,30 l'SVD) ma un comportamento profondamente diverso, e vale la pena guardarlo. Sui 2,5 milioni di celle riempite il K-NN produce appena 137.097 valori distinti, l'SVD 1.744.633: dodici volte tanti. Il motivo è che il K-NN, per stimare quanto un utente voterà un film, ha bisogno che quel film abbia dei vicini fra i titoli che l'utente ha già votato; quando non li trova non può dire nulla e ripiega sulla media globale del training set, sempre la stessa per tutti. Su una matrice riempita al 99,7% di celle vuote succede continuamente. L'SVD invece un valore ce l'ha sempre, perché lo calcola dai fattori latenti, e infatti differenzia quasi ogni singola coppia utente-film. È lo stesso limite strutturale che si legge nell'RMSE, visto qui dal lato del riempimento. Il confronto è quello della tabella qui sopra: il K-NN nella sua configurazione migliore si ferma a RMSE 1,0630 e MSE 1,1299, la matrix factorization arriva a 0,9401 e 0,8838. Per questo le liste di raccomandazione del paragrafo successivo e la segmentazione degli utenti le costruisco sul riempimento dell'SVD, tenendo quello del K-NN come termine di paragone.

4.4 Le liste Top-N

Il filling trasforma il modello in un sistema di raccomandazione: per ogni utente ordino i voti previsti sui film non ancora visti e prendo i primi dieci. Un dettaglio a cui ho dovuto fare attenzione: le previsioni di Surprise sono normalmente “tagliate” nell'intervallo 1-5, e per gli utenti generosi moltissimi film finirebbero a 5,00 esatto, una classifica di pari merito, di fatto casuale. Per ordinare in modo sensato uso le stime non troncate (clip=False): il punteggio grezzo varia da film a film e produce una classifica reale. Le liste generate (top-10 per 2.000 utenti, 20.000 raccomandazioni salvate) mostrano infatti punteggi che superano regolarmente il tetto delle cinque stelle (il primo titolo consigliato a un utente arriva a 5,396), cosa impossibile con il troncamento attivo. Ho poi verificato quanto spesso rimangano punteggi identici in una stessa lista: nel 66,6% dei casi i dieci punteggi risultano tutti distinti già alla terza cifra decimale, che è la precisione con cui vengono salvati su file, mentre l'ordinamento avviene sempre sul valore a piena precisione. Anche l'estrazione degli utenti e dei candidati usa un generatore casuale a seme fisso, quindi le liste sono identiche a ogni riesecuzione. Per contenere il costo del calcolo quando i film non ancora valutati da un utente superano i cinquemila i candidati vengono estratti a caso fra questi. La lista è quindi la migliore decina di un sottoinsieme ampio e casuale del catalogo, non dell'intero catalogo; è una scelta che non cambia la natura della procedura, ma che incide sulle singole liste.

4.5 Segmentazione degli utenti con K-Means

L'ultimo passo raggruppa gli utenti in segmenti simili. Anche qui bisogna rispondere a due domande: come rappresentare un utente, e in quanti gruppi dividerli. Per la prima: i voti osservati non funzionano (ognuno ha visto film diversi, i vettori non sono confrontabili), quindi uso il profilo ricostruito dal filling, cioè la riga dell'utente nella matrice completata, densa per costruzione. Questa riga la comprimo a 20 dimensioni con la PCA per togliere rumore e la normalizzo con la L2, in modo che il K-Means lavori di fatto con la similarità coseno. Per la seconda: il numero di gruppi "l'ho chiesto" ai dati confrontando, per k da 2 a 8, l'Elbow (quanto i gruppi sono compatti) e la Silhouette (quanto sono separati: va da -1 a 1 , più è alta meglio è).

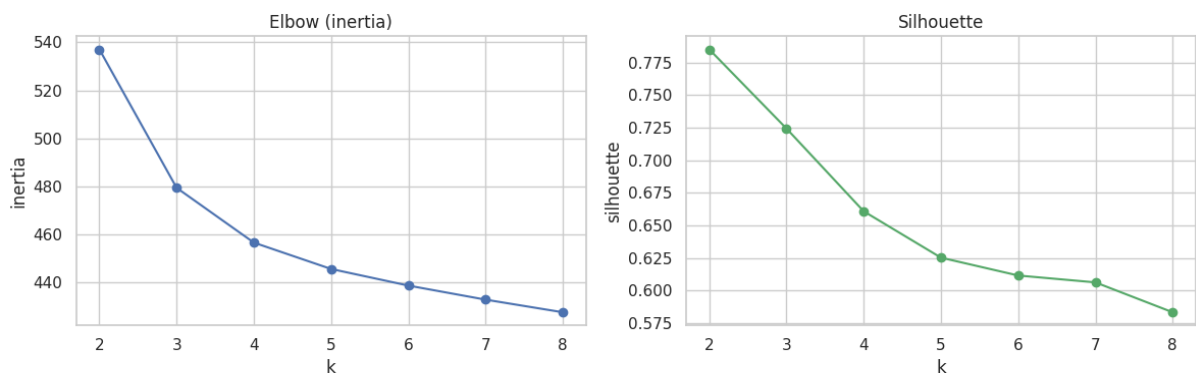


Figura 4.3. Scelta di k : Elbow (inertia) e Silhouette per $k=2..8$.

Il massimo della silhouette è a $k=2$, con un valore di 0,785: alto, segno di due gruppi realmente separati. E i gruppi sono anche bilanciati, 1.718 utenti nel primo e 2.282 nel secondo, non un cluster gigante con pochi esclusi.

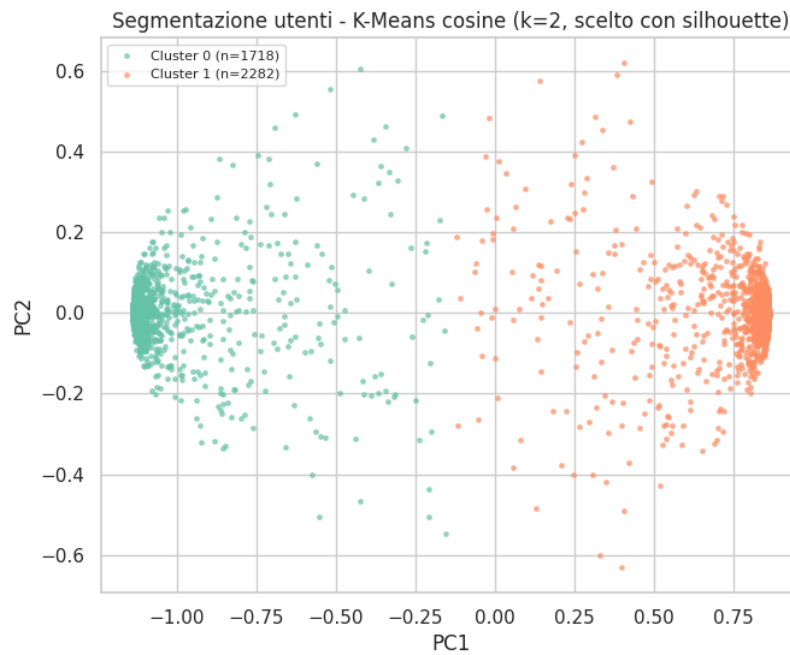


Figura 4.4. Proiezione PCA 2D dei due cluster di utenti.

Cluster	Utenti	Rating	Voto medio	Interpretazione
0	1.718	64.819	3,69	utenti "critici"
1	2.282	77.044	4,67	utenti "generosi"

L'interpretazione è arrivata dalle statistiche dei gruppi, ed è netta: la segmentazione separa gli utenti per stile di voto, con quasi un punto di differenza tra le medie. È lo sbilanciamento visto nell'EDA, ma il clustering mostra che non è uniforme: esistono "critici" e "generosi", in proporzioni simili. In ottica business è un'informazione concreta: un 4 dato da un critico vale più di un 4 dato da un generoso, e le soglie per decidere se raccomandare andrebbero calibrate sui due gruppi.

5. Parte intermedia, Content-Based Filtering

Il content-based cambia la fonte del segnale: invece di chiedersi “cosa hanno votato gli utenti simili a te”, si chiede “che cosa contiene questo film, e quanto somiglia a quelli che ti sono piaciuti”. Il suo vantaggio decisivo è il cold-start: un film appena uscito, senza voti, è invisibile al collaborative filtering ma perfettamente descrivibile dal suo contenuto.

5.1 Metadati e preprocessing

Come materiale testuale uso titolo e descrizione dei film, presi dai metadati del dataset. Il caricamento ha richiesto un accorgimento: il file ha un campo annidato con struttura incoerente (a volte lista, a volte stringa) che manda in errore il lettore automatico a tabella; l'ho letto riga per riga con il parser json, tenendo solo i campi utili. Tutti i 12.500 film del dataset filtrato hanno testo utilizzabile (copertura 100%). Sul testo destinato al TF-IDF applico la tokenizzazione (spezzare il testo in parole), rimozione delle stopwords (parole comunissime come “the” e “and”, che non distinguono un testo dall'altro) e lemmatizzazione (ricondurre le parole alla forma base).

5.2 I due embedding: TF-IDF e DistilBERT

Per confrontare i testi bisogna trasformarli in vettori di numeri (embedding). Ho costruito le due rappresentazioni richieste, che incarnano filosofie opposte. Il TF-IDF (20.000 termini) è quella “a frequenza”: descrive ogni film in base a quali parole usa e quanto sono rare e distintive, “samurai” pesa molto, “movie” quasi nulla. DistilBERT (via transformers, con mean-pooling degli stati nascosti) è quella neurale: una rete pre-addestrata che comprime il significato del testo in 768 numeri.

Ho deciso di dare al TF-IDF il testo pulito, ma al transformer il testo grezzo. Il motivo è che i transformer sono addestrati sul linguaggio naturale vero: togliere stopwords e coniugazioni li priva proprio del contesto che li rende forti, mentre il TF-IDF dal testo pulito ci guadagna. Dare a entrambi lo stesso input sarebbe stato più “simmetrico”, ma avrebbe penalizzato il neurale.

5.3 Predizione dei rating e confronto tra embedding

Una volta ottenuti gli embedding, il passo successivo è usarli per predire i voti dei film e confrontare i tre approcci sulla stessa metrica. La predizione avviene con un K-NN per utente: le feature sono gli embedding dei film che ha valutato e le etichette sono i suoi voti. Un KNeighborsRegressor con metrica coseno e pesi sulla distanza prevede i voti dei film di test come media pesata dei voti dati ai film più simili. La valutazione usa un unico split 80/20 condiviso da tutti gli embedding e copre l'intero test set: 28.373 coppie utente-film. Così le differenze tra embedding, che si riveleranno piccolissime, non possono dipendere dal campione.

Embedding	RMSE	MSE	Coppie di test
Transformer (DistilBERT)	0,9835	0,9673	28.373
TF-IDF	0,9836	0,9674	28.373

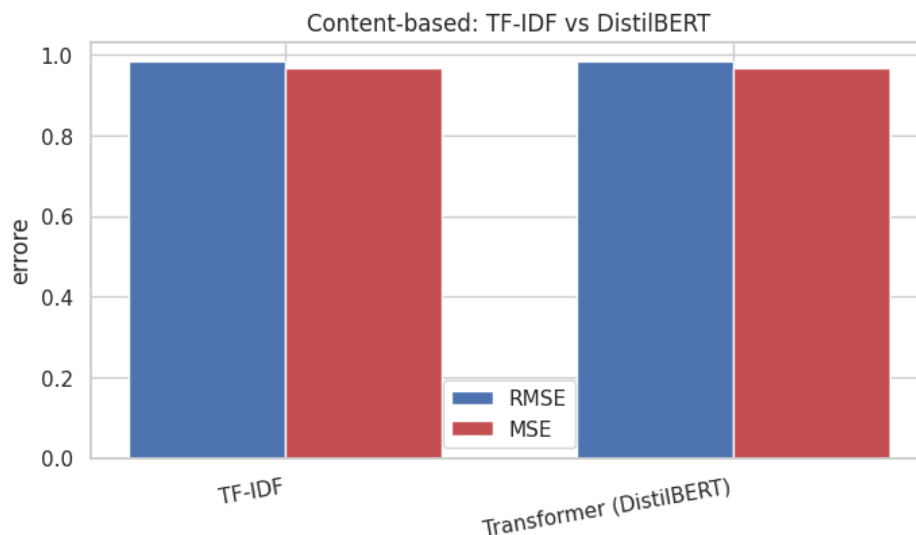


Figura 5.1. Confronto tra gli embedding sul test set completo.

Il risultato è un pareggio sostanziale: 0,9835 contro 0,9836, una differenza sulla quarta cifra decimale. Il pareggio ha una lettura precisa: sui metadati dei film, il conteggio pesato delle parole e la comprensione semantica del transformer arrivano allo stesso livello, il TF-IDF cattura i termini distintivi (attori, saghe, generi), DistilBERT compensa con i sinonimi e il contesto, e nessuno dei due ha un vantaggio decisivo. Nel capitolo 8.3 ho verificato questa lettura con un terzo embedding specializzato che conferma il quadro.

5.4 Content-based contro collaborative filtering

Per il confronto finale tra i due paradigmi ho evitato di paragonare l'RMSE del content-based su questo split con quello della cross-validation della parte base, questo infatti avrebbe confrontato numeri nati da protocolli diversi. Ho invece riallenato il collaborative filtering sullo stesso split 80/20 e l'ho valutato sulle stesse 28.373 coppie di test: stessi utenti, stessi film, stesso protocollo. Il modello di confronto è l'SVD di Surprise con i parametri predefiniti, e non con la configurazione vincente della grid search precedente. È una scelta conservativa: usare i parametri di default penalizza il CF nel confronto, quindi se il content-based dovesse comunque perdere, la sconfitta sarebbe meno equivoca; e se il CF vince, la vittoria è ancora più solida perché ottenuta senza tuning. Sullo stesso split riporto per completezza anche il K-NN item-based con similarità coseno, che si ferma a RMSE 1,0710: è l'unico dei quattro modelli confrontati a restare dietro al content-based, e conferma che il vantaggio del collaborative filtering viene dalla fattorizzazione latente, non dal paradigma in quanto tale.

Approccio	RMSE	MSE
Collaborative Filtering (SVD, stesso split)	0,9498	0,9020
Content-Based (DistilBERT)	0,9835	0,9673
Content-Based (TF-IDF)	0,9836	0,9674
Collaborative Filtering (KNNBasic, stesso split)	1,0710	1,1470

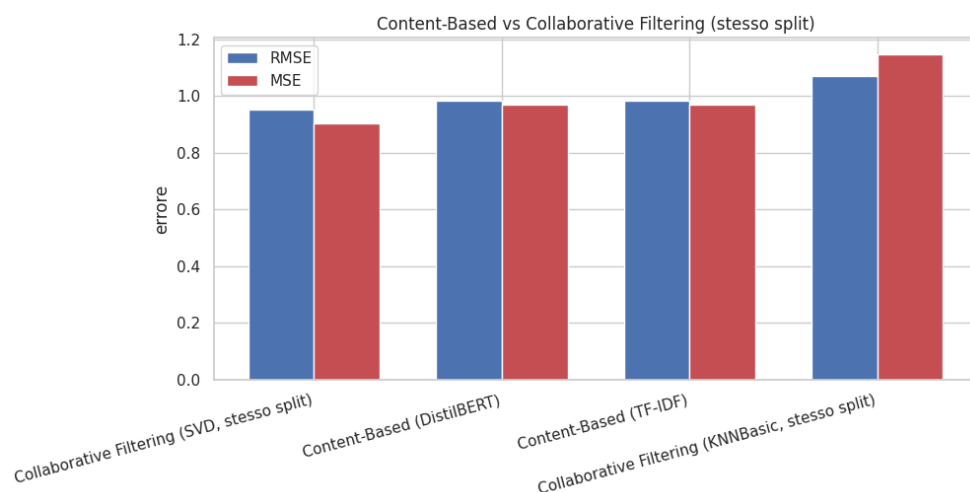


Figura 5.2. CB vs CF sullo stesso split e sulle stesse coppie di test.

Il collaborative filtering vince con un margine di circa 0,034 di RMSE. Sapere che cosa è piaciuto alle persone con i miei stessi gusti è un segnale più forte della somiglianza di contenuto con ciò che ho già visto. Il content-based però non esce ridimensionato, perché il distacco è di appena tre centesimi, e resta l'unico dei due a funzionare sui film senza storia. La conclusione operativa è la complementarità: il CF come motore principale, il content-based per il cold-start.

6. Parte avanzata, Sentiment Analysis

La terza parte cambia prospettiva: non prevedere voti, ma leggere il testo di una recensione e capirne il giudizio. È un problema di classificazione a tre classi, con le etichette ricavate dal voto come da traccia: 1-2 stelle → negativo, 3 → neutro, 4-5 → positivo.

6.1 Etichette, sbilanciamento e campione

Sul campione da un milione la distribuzione è: 795.715 positive (79,6%), 114.946 negative (11,5%) e appena 89.339 neutre (8,9%). Questo squilibrio crea una trappola nella metrica più ovvia, l'accuracy (la percentuale di risposte giuste): un modello pigro che rispondesse sempre “positivo” azzeccerebbe l'80% sembrando bravo, senza aver capito nulla. Per questo la metrica principale è la F1 macro: fa la media delle prestazioni sulle tre classi dando a ciascuna lo stesso peso, così la classe neutro (piccola) conta quanto la positiva (enorme), e chi ignora le classi rare viene punito. Prima di procedere, un chiarimento sui dati usati in questa parte: la parte avanzata riparte da un secondo passaggio in streaming sul milione di recensioni, questa volta tenendo i soli campi che le servono (voto, titolo e testo) e senza applicare la deduplicazione descritta all'inizio del report, perché qui ogni recensione vale come esempio a sé e due giudizi dello stesso utente sullo stesso film non falsano nulla. Dato che i duplicati erano lo 0,26%, le proporzioni fra le classi sono le stesse delle 997.406 recensioni pulite viste nell'analisi esplorativa.

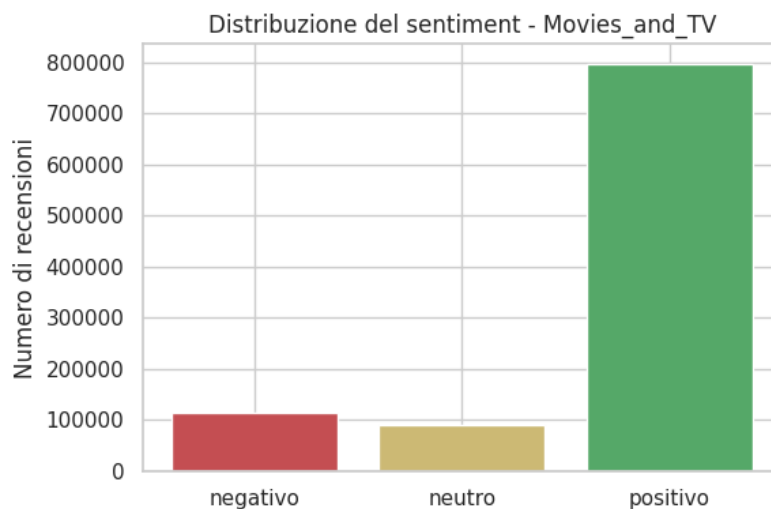


Figura 6.1. Distribuzione delle classi di sentiment.

Per l'addestramento ho estratto un campione stratificato di 40.001 recensioni (il numero non è tondo perché le quote delle tre classi vengono arrotondate una per una). Stratificato significa che le proporzioni tra le classi restano quelle vere, così la difficoltà del problema non viene alterata. Ho unito titolo e corpo della recensione, perché il titolo (“Waste of money”, “Amazing!”) contiene spesso il giudizio già concentrato. Dopo la pulizia dei testi rimasti vuoti restano 39.900 recensioni: 31.920 di training e 7.980 di test, con split stratificato.

6.2 Le feature: TF-IDF e transformer

Come nella parte intermedia, due rappresentazioni: TF-IDF a 10.000 termini sul testo pre-processato, e DistilBERT sul testo grezzo (stessa logica: il transformer lavora meglio sul linguaggio naturale). Le feature dense del transformer passano per uno StandardScaler addestrato solo sul training set, mai sul test, per non far filtrare informazione.

6.3 I classificatori e la gestione dello sbilanciamento

Ho messo alla prova i quattro classificatori (Decision Tree, Random Forest, Gaussian Naive Bayes e K-Nearest Neighbors) su entrambe le rappresentazioni. Contro lo sbilanciamento ho usato due strumenti: `class_weight="balanced"` dove il modello lo supporta (gli errori sulle classi rare pesano di più durante l'addestramento) e l'oversampling con `RandomOverSampler` dove non lo supporta (gli esempi delle classi rare vengono ricampionati fino a pareggiare la dominante, solo nel training, mai nel test). Il Naive Bayes gaussiano richiede dati densi, quindi sul TF-IDF sparso ho usato gli altri tre.

Classificatore	Accuracy	Precision (macro)	Recall (macro)	F1 macro
Random Forest (TF-IDF)	0,861	0,870	0,553	0,621
Decision Tree (TF-IDF)	0,785	0,571	0,608	0,587
KNN k=5 (Transformer)	0,731	0,529	0,617	0,552
Decision Tree (Transformer)	0,767	0,522	0,523	0,522
Gaussian NB (Transformer)	0,660	0,510	0,628	0,522
Random Forest (Transformer)	0,830	0,858	0,440	0,473
KNN k=5 (TF-IDF)	0,501	0,452	0,511	0,397

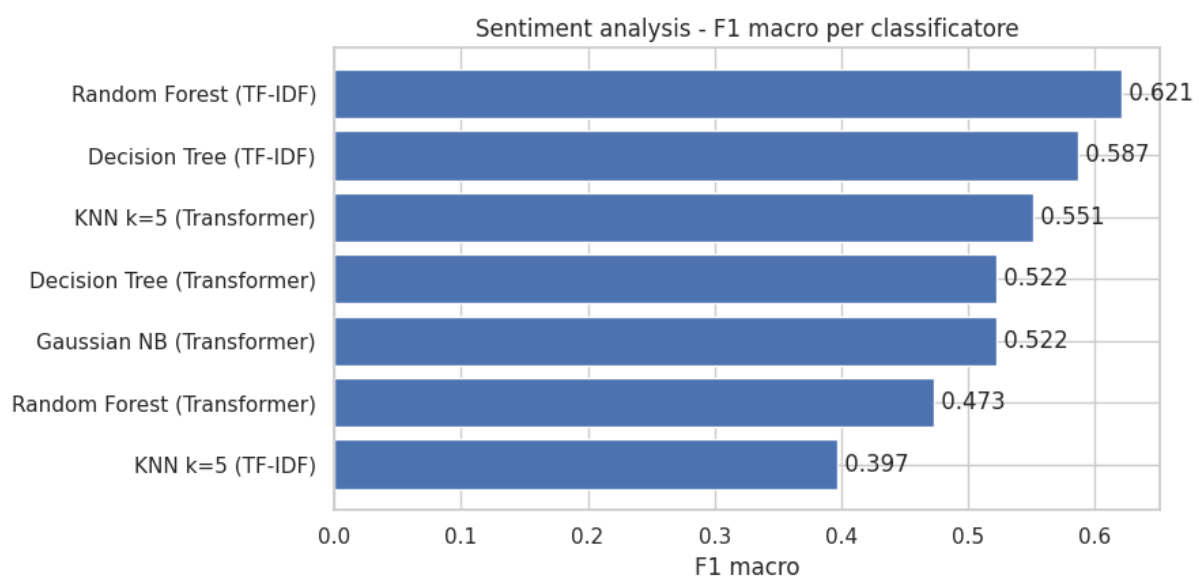


Figura 6.2. Confronto dei classificatori per F1 macro.

Il migliore è il Random Forest su TF-IDF: F1 macro 0,621, accuracy 0,861. Da questa classifica emergono due elementi. Il primo: sul sentiment il TF-IDF resta avanti per i modelli ad albero, perché il lessico

esplicito delle recensioni (“terrible”, “masterpiece”) è un indizio fortissimo che le frequenze pesate catturano alla perfezione. Al contrario gli embedding del transformer funzionano meglio con il K-NN (0,552 contro 0,397), che ragiona su distanze in uno spazio denso. Conta quindi l'accoppiata embedding-classificatore, non l'embedding da solo. Il secondo: il crollo del K-NN su TF-IDF ha una ragione geometrica, cioè che in 10.000 dimensioni sparse le distanze tra punti diventano tutte simili e il concetto di “vicino” perde significato.

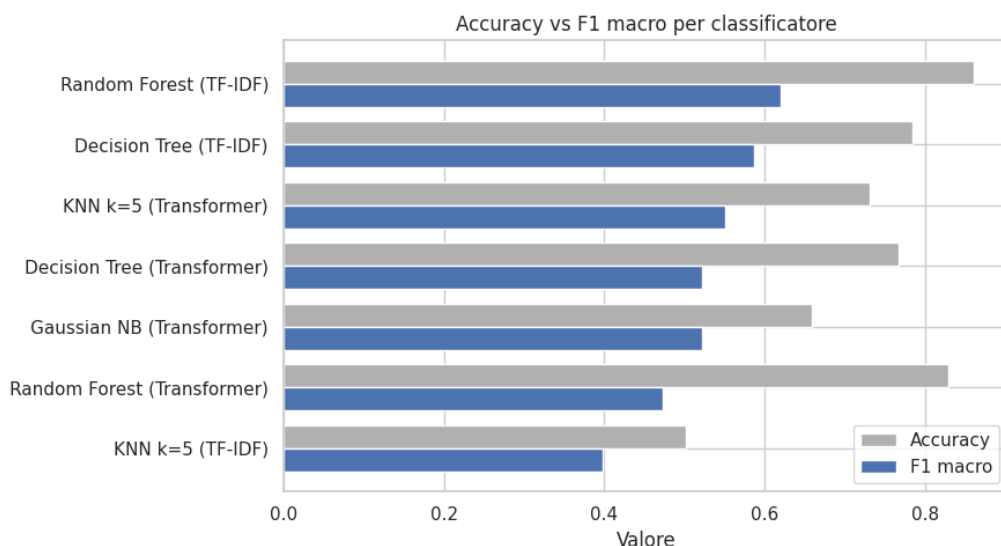


Figura 6.3. Accuracy vs F1 macro: l'effetto dello sbilanciamento.

Il grafico che affianca accuracy e F1 macro è la fotografia dello sbilanciamento: quasi tutti i modelli hanno un divario ampio tra le due metriche, perché azzeccano la classe dominante (accuracy alta) e sbagliano le rare (F1 bassa). È esattamente il motivo per cui ho scelto la F1 macro come metrica di riferimento.

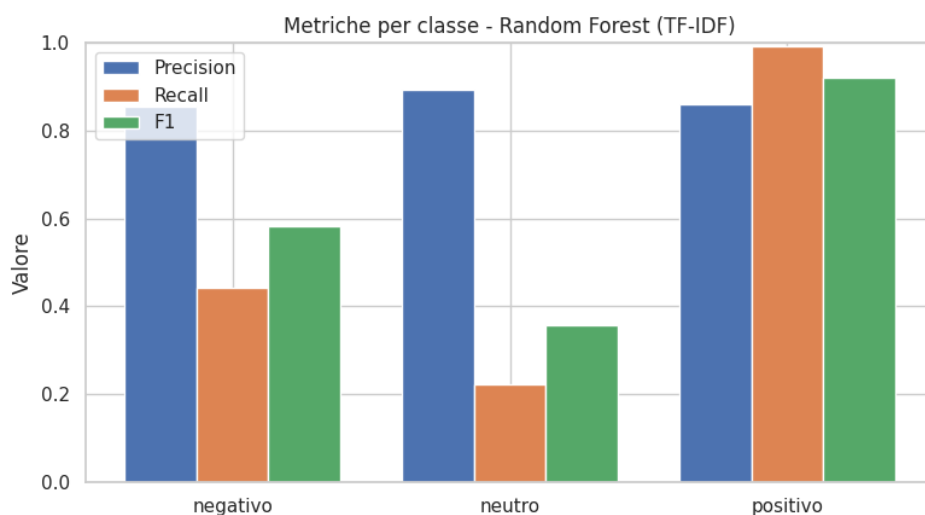


Figura 6.4. Metriche per classe del modello migliore.

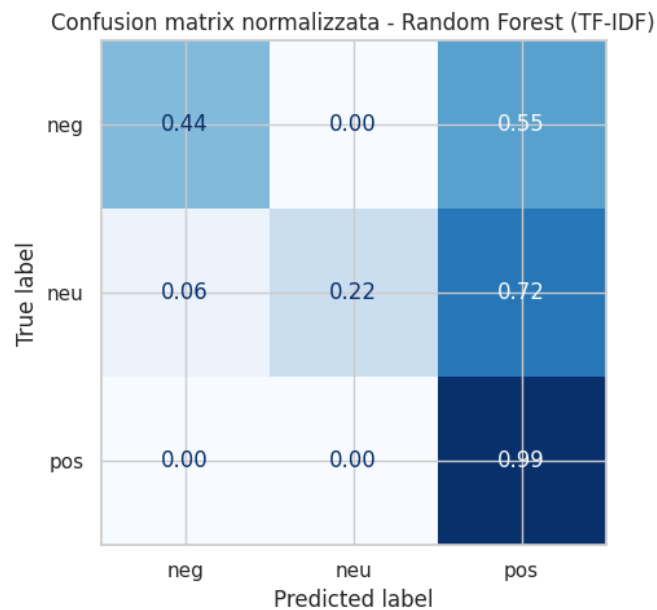


Figura 6.5. Confusion matrix normalizzata (ogni riga somma a 1).

Guardando dentro il modello migliore classe per classe, il quadro diventa concreto. La classe positiva è quasi perfetta: F1 0,92 e recall 0,99, cioè il modello prende 99 recensioni positive su 100. Non stupisce: sono l'80% dei dati e usano un lessico riconoscibile (loved, amazing, favorite, must watch). La classe negativa sta in mezzo: F1 0,58 e recall 0,44, meno della metà delle recensioni negative viene individuata. Il modello ne cattura solo la parte più esplicita, quella con parole di segno chiaro (terrible, waste, boring), e perde le altre. La classe neutra è la vera criticità: F1 0,36 e recall 0,22, cioè quasi quattro recensioni neutre su cinque finiscono classificate altrove, e la confusion matrix mostra che quasi tutte scivolano nella classe positiva.

La ragione è duplice, statistica e linguistica. Sul piano statistico il neutro è raro (~9% del training) e i modelli, penalizzando poco l'errore su una classe minoritaria, imparano a "non scommettere" sul neutro. Sul piano linguistico una recensione da 3 stelle usa spesso lo stesso vocabolario di una da 4: carino ma..., non male ma..., mi aspettavo di più. Il segnale che distingue le due è nella tenuta di quel "ma", in una sfumatura che il classificatore bag-of-words fatica a cogliere. Le due cause si sommano: pochi esempi e linguaggio ambiguo. Il risultato è che il modello, quando è incerto, punta sulla classe più probabile a priori, cioè la positiva e sbaglia sistematicamente sul neutro.

Su questa classe torno più avanti, nelle analisi aggiuntive, con un esperimento mirato che sostituisce i classificatori ad albero con modelli lineari `class_weight="balanced"` per verificare se il problema sia risolvibile senza toccare le feature.

7. Bonus, Sentiment con un LLM (Groq)

Fino a qui il sentiment è stato affrontato con modelli addestrati sui dati. Ma con la diffusione dei modelli linguistici pre-addestrati di grandi dimensioni vale la pena rispondere a una domanda diversa: un LLM generalista, che non ha mai visto le mie recensioni, sa classificare il sentiment quanto un modello addestrato apposta? Per rispondere ho preso 300 recensioni del test e le ho classificate una a una con un LLM servito da Groq tramite langchain-groq (ChatGroq, modello openai/gpt-oss-20b, temperature=0 per risposte deterministiche), con un prompt che chiede di rispondere solo con l'etichetta. Le 300 recensioni sono estratte dal test set con un generatore casuale a seme fisso, quindi il sottoinsieme è lo stesso a ogni esecuzione. Il confronto è con il Random Forest, valutato sulle stesse identiche 300 recensioni: non lo riaddestro, riuso le predizioni che aveva già prodotto sull'intero test set e ne ritaglio quelle 300.

Durante lo svolgimento sono emersi tre accorgimenti che si sono rivelati necessari: passare all'LLM il testo grezzo (il modello è addestrato sul linguaggio naturale, il preprocessing lo danneggia); gestire i modelli 'reasoning', che nel ragionamento citano più etichette, motivo per cui il parser estrae l'ultima menzionata, che è la conclusione effettiva; e inserire una pausa tra le chiamate per rispettare i limiti del piano gratuito, con un secondo tentativo in caso di errore e, se anche questo fallisce, l'assegnazione dell'etichetta neutra come valore di ripiego.

Approccio	Accuracy	F1 macro
LLM (Groq, openai/gpt-oss-20b)	0,890	0,754
Classificatore classico (Random Forest)	0,833	0,572

Bonus LLM - confronto su 300 recensioni

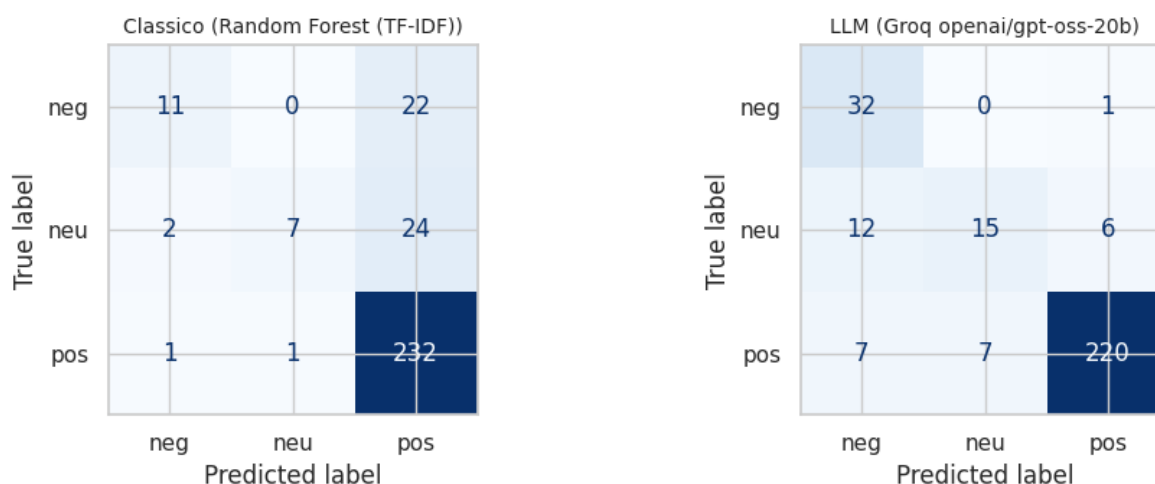


Figura 7.1. LLM vs classificatore classico sulle stesse 300 recensioni.

Il risultato ottenuto è che l'LLM vince, e non di poco sulla metrica difficile, F1 macro 0,754 contro 0,572, accuracy 0,890 contro 0,833. Un modello che vede queste recensioni per la prima volta, in modalità zero-shot, batte un classificatore addestrato su 31.920 esempi del dominio. Il rovescio della medaglia è pratico: il Random Forest, una volta addestrato, classifica le 7.980 recensioni del test in pochi secondi

(l'addestramento ne richiede 91), l'LLM ha impiegato 12.9 minuti per sole 300 recensioni per via dei limiti dell'API. Un'avvertenza sulla lettura del confronto: su queste 300 recensioni il Random Forest ottiene una F1 macro (0,572) più bassa di quella che realizza sull'intero test set (0,621), semplicemente perché trecento esempi sono pochi e le classi rare vi compaiono in manciate di unità. Il divario a favore dell'LLM va quindi preso come un'indicazione di direzione, non come una misura precisa. La mia conclusione è che, dove le etichette ci sono e i volumi contano, il classificatore addestrato resta la scelta pratica; ma la qualità dell'LLM senza addestramento indica chiaramente la direzione che questo campo sta prendendo,

8. Analisi aggiuntive oltre il programma

Durante lo sviluppo mi sono venuti diversi dubbi del tipo “e se avessi usato...?”. Invece di lasciarli cadere, ho preparato una versione estesa del notebook in cui li ho messi alla prova in modo controllato: stessi dati, stessi split, stesso protocollo del progetto principale, così ogni differenza nei numeri è attribuibile solo alla tecnica in esame. Quattro i test: tre tecniche oltre il programma del corso e un esperimento sul dimensionamento del dataset. Per ciascuno riporto il ragionamento, il risultato e la conclusione; nei grafici il blu indica i metodi del corso, l'arancio le estensioni.

8.1 KNNBaseline: il K-NN con correzione dei bias

Il ragionamento parte da un risultato del progetto stesso: il clustering ha mostrato che esistono “critici” e “generosi”, con quasi un punto di differenza nel voto medio. Il KNNBasic ignora questa informazione: un 4 vale 4, chiunque lo dia. La variante KNNBaseline prima stima quanto ogni utente è generoso e quanto ogni film è amato (i “bias”), poi confronta gli utenti sugli scostamenti dal proprio stile, non sui voti assoluti.

Il risultato conferma: RMSE 1,0021 contro 1,0630 del KNNBasic, un miglioramento di 0,061. Non abbastanza, però, per insidiare l'SVD (0,9401), che i bias li incorpora già nella sua formulazione.

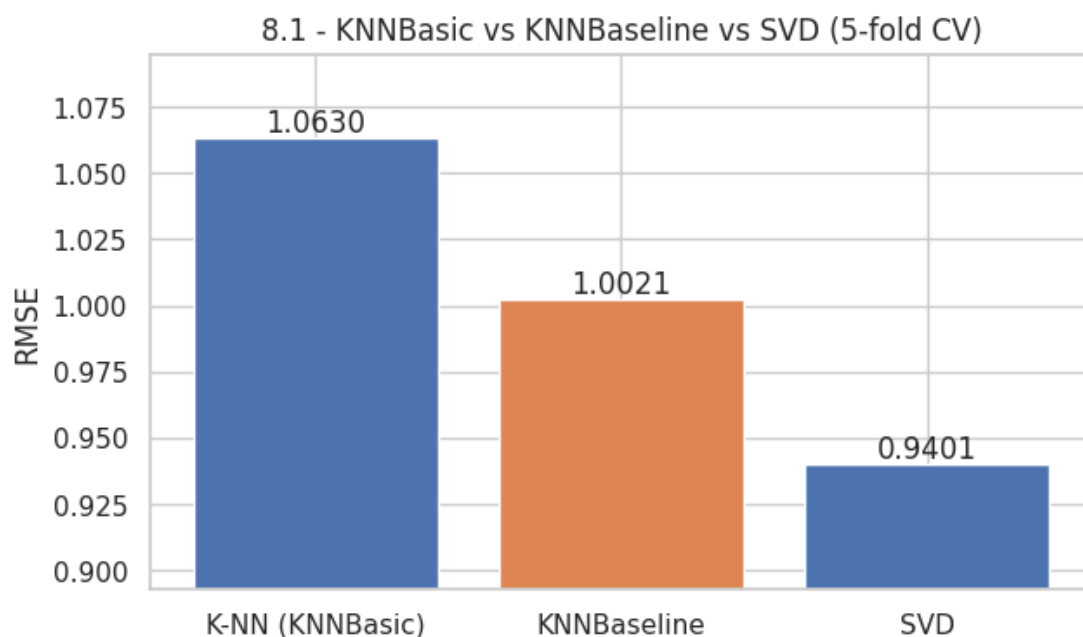


Figura 8.1. RMSE dei tre modelli CF: la correzione dei bias migliora il K-NN, ma l'SVD resta il migliore.

Conclusione: l'estensione funziona e conferma che i bias di voto sono un fenomeno reale di questi dati (in coerenza col clustering).

8.2 SVD sul k-core completo: mettere alla prova i tetti

Questo test nasce da un dubbio sul mio stesso setup. I tetti su utenti e prodotti servono al K-NN, ma l'SVD non ha quel vincolo di memoria: avrei potuto allenarlo sul k-core completo, cioè su tutti i 33.698 utenti e i 30.923 titoli sopravvissuti al filtro. Tenerlo sul dataset ridotto significava sprecare dati? L'intuizione dice che più dati è meglio; l'ho verificato rieseguendo la cross-validation dell'SVD, con la stessa configurazione vincente, sul k-core completo (463.509 voti, senza nessuno dei due tetti).

Il risultato mi ha sorpreso: l'RMSE peggiora, da 0,9401 a 0,9687. La spiegazione è nella densità: i 29.698 utenti e i 18.423 titoli rimessi dentro sono i meno attivi e i meno recensiti, la densità della matrice crolla dallo 0,284% allo 0,045%, e per quegli utenti il modello ha troppo pochi voti per stimare fattori affidabili. Più dati, ma più poveri.

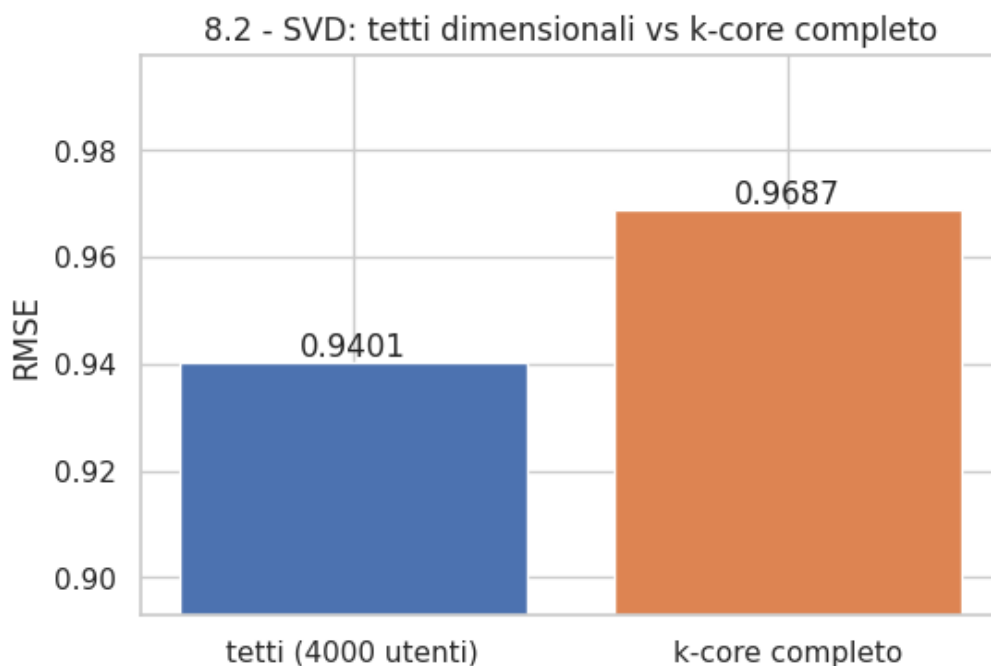


Figura 8.2. SVD con cap utenti vs SVD sul k-core completo: più utenti, risultato peggiore.

Questo test trasforma una scelta nata come compromesso di memoria in una scelta giustificata sperimentalmente. Il dataset con i tetti non è solo più leggero, è anche quello su cui il modello lavora meglio.

8.3 SBERT: un embedding specializzato nella similarità

Nella parte intermedia TF-IDF e DistilBERT sono arrivati alla pari. Per capire se un transformer costruito apposta per confrontare testi potesse fare meglio, ho aggiunto un terzo embedding: sentence-transformers all-mpnet-base-v2, valutato sulle stesse 28.373 coppie.

Risultato: RMSE 0,9837, in linea perfetta con gli altri due (0,9835-0,9836). Nemmeno la specializzazione sposta l'ago: su questi metadati l'informazione utile che il testo contiene è evidentemente tutta lì, e le tre rappresentazioni la estraggono per intero, in modi diversi.

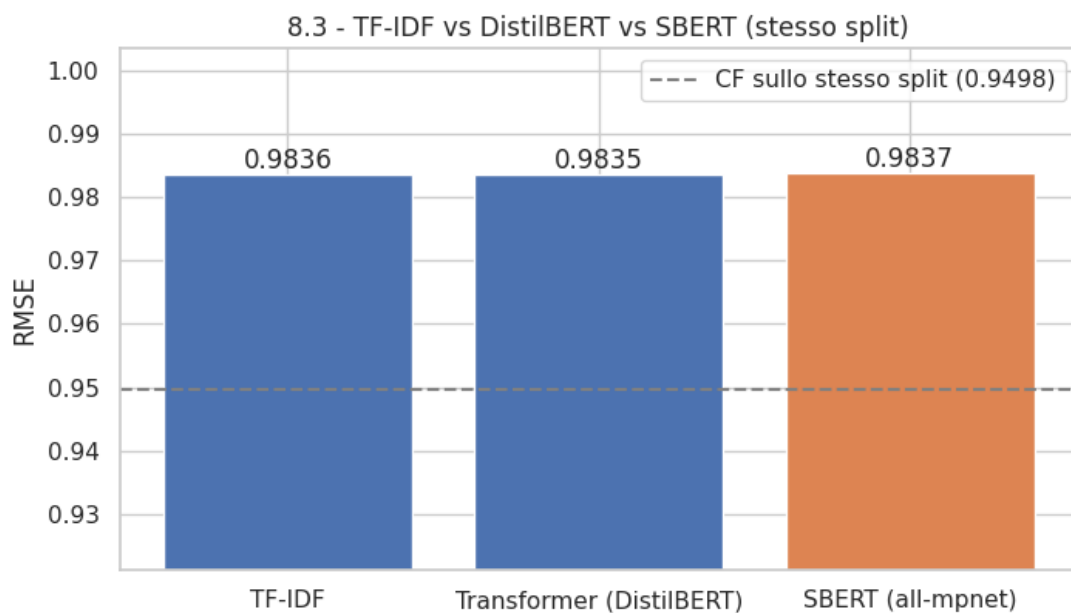


Figura 8.3. I tre embedding a confronto; la linea tratteggiata è il riferimento del CF sullo stesso split.

Possiamo dunque concludere che il limite del content-based su questa categoria non è la qualità dell'embedding, è il contenuto stesso, e il divario con il CF (0,9498) non si colma cambiando rappresentazione.

8.4 Classificatori lineari: LinearSVC e Logistic Regression

L'ultimo esperimento riguarda la classe che il progetto principale non doma: il neutro. Per un problema come questo, dove la classe rara è proprio quella che i modelli precedenti hanno mancato di più, un classificatore lineare con `class_weight='balanced'` ha il vantaggio di penalizzare esplicitamente gli errori sulla classe minoritaria, un correttivo che nel Random Forest è molto più difficile da imporre. Ho aggiunto LinearSVC e Logistic Regression, a parità di feature, split e correttivi. Il risultato è netto. Il LinearSVC sugli embedding del transformer raggiunge F1 macro 0,708, un salto di +0,087 rispetto al Random Forest; seguono il LinearSVC su TF-IDF (0,689) e le due Logistic Regression (0,681 su TF-IDF e 0,671 sul transformer). Tutte e quattro le estensioni superano tutti i modelli del corso. Il guadagno arriva proprio dalle classi difficili: la recall del neutro sale da 0,22 a 0,51 (più che raddoppiata) e quella del negativo da 0,44 a 0,77. Vale la pena notare anche il tempo di addestramento: il LinearSVC su TF-IDF si addestra in 1 secondo, contro i 94 del Random Forest. Il divario si spiega con la natura degli algoritmi: il LinearSVC risolve un unico problema di ottimizzazione convessa che sfrutta la sparsità di TF-IDF, mentre il Random Forest costruisce centinaia di alberi indipendenti cercando su ciascuno lo split migliore fra migliaia di feature. Il modello migliore, quindi, è anche il più economico da addestrare.

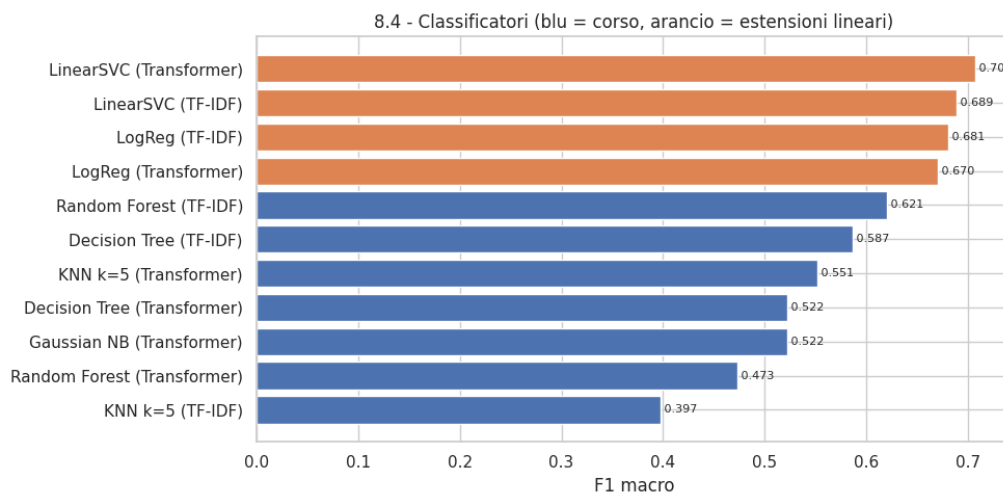


Figura 8.4. F1 macro dei classificatori: le estensioni lineari superano tutti i modelli del corso.

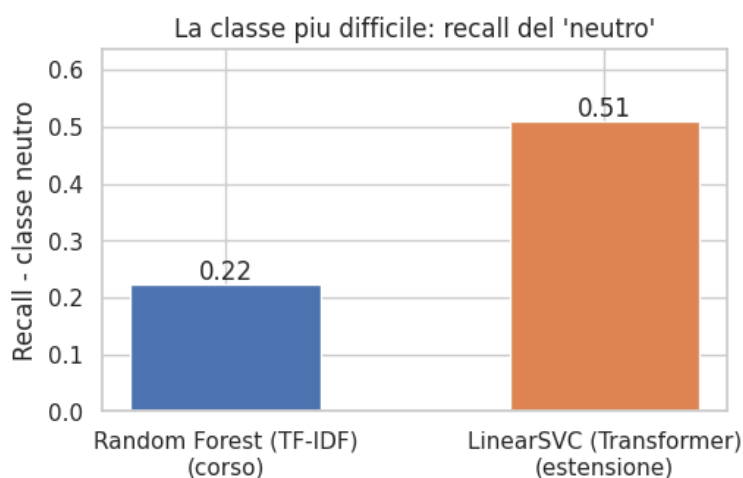


Figura 8.5. Recall della classe neutro: il miglioramento più significativo.

Sui testi, un modello lineare ben pesato batte modelli più complessi, e vince proprio sulla classe difficile.

8.5 Il quadro complessivo dei test

Test (versione estesa)	Metrica	Corso	Esteso	Esito
KNNBaseline vs KNNBasic	RMSE	1,0630	1,0021	migliora il K-NN, non batte SVD
SVD sul k-core completo	RMSE	0,9401	0,9687	peggiora: tetti giustificati
SBERT vs TF-IDF/DistilBERT	RMSE	0,9836	0,9837	equivalente: il limite è nei dati
LinearSVC vs Random Forest	F1 macro	0,621	0,708	miglioramento significativo

Un'estensione migliora davvero (LinearSVC), una migliora senza cambiare le decisioni (KNNBaseline), due confermano le scelte del progetto principale (il cap sugli utenti, l'equivalenza degli embedding). Non tutte le estensioni hanno migliorato il progetto, ma anche i test negativi hanno un valore: trasformano le scelte iniziali da assunzioni in decisioni verificate.

9. Limiti e scelte metodologiche

Ogni scelta progettuale porta con sé dei limiti; qui li richiamo per completezza, così che i risultati possano essere letti nel loro contesto.

Il campionamento. Dei circa 17 milioni di recensioni ne uso un milione, estratte in streaming con mescolamento e seed fisso: un campione casuale e riproducibile, le statistiche vanno lette come stime riferite ad esso.

I tetti dimensionali. I limiti a 4.000 utenti e 12.500 prodotti nascono dal costo quadratico delle matrici di similarità del K-NN (il tetto sui prodotti l'ho dimensionato empiricamente, dopo che la versione senza tetto ha esaurito la RAM). Concentrano l'analisi sugli utenti attivi e sui titoli popolari; il test 8.2 mostra però che su questo dataset sono anche la scelta più accurata per l'SVD, non solo quella obbligata.

I protocolli di confronto. Tutti i confronti su cui baso conclusioni (embedding tra loro, content-based contro collaborative, LLM contro classico, estensioni del corso) avvengono sugli stessi split e sulle stesse coppie o sottoinsiemi di test. L'unica differenza di configurazione, dichiarata nel capitolo 5.4, riguarda gli iperparametri dell'SVD usato come riferimento sullo split condiviso, ed è una differenza che penalizza il collaborative filtering. Nessun numero confrontato in questo report proviene da protocolli diversi.

Il clustering. La silhouette di 0,785 è misurata sui profili ricostruiti dal modello e compressi con la PCA: indica separazione in quello spazio, che la struttura appresa dall'SVD favorisce. L'interpretazione dei gruppi (stile di voto) è comunque coerente anche con le statistiche osservate.

Il bonus LLM. Le 300 recensioni sono il dimensionamento necessario per i limiti dell'API gratuita; un campione più ampio darebbe stime più stabili, e il vantaggio dell'LLM osservato qui andrebbe confermato su scala maggiore.

10. Conclusioni

Dal lavoro emergono tre risultati principali. Il primo riguarda il collaborative filtering: la matrix factorization batte nettamente il K-NN (RMSE 0,9401 contro 1,0630) perché sui dati sparsi i fattori latenti generalizzano dove i vicini osservati non bastano, e nemmeno la correzione dei bias del K-NN (test 8.1) colma il divario. La segmentazione K-Means aggiunge un risultato leggibile: due profili di voto ben separati, “critici” e “generosi”, che suggeriscono di calibrare le raccomandazioni sullo stile di voto.

Il secondo riguarda il confronto tra paradigmi: a parità di protocollo il collaborative filtering supera il content-based (0,9498 contro 0,9835), ma il distacco è contenuto e il content-based resta l'unica risposta al cold-start; quanto agli embedding, dando al transformer il testo grezzo le due rappresentazioni si equivalgono, e nemmeno un modello specializzato (test 8.3) cambia il quadro: il limite sta nei dati, non nella rappresentazione.

Il terzo riguarda il sentiment: lo sbilanciamento è il vero avversario, perché la classe neutro resta la più difficile per ogni modello, e la F1 macro si è confermata la metrica onesta. Il dato più moderno del progetto è il bonus: l'LLM zero-shot batte il classificatore addestrato (F1 0,754 contro 0,572), pur restando impraticabile su grandi volumi per i tempi di risposta.

Il lavoro apre a tre direzioni di sviluppo naturali: un sistema ibrido collaborative+content per coprire il cold-start senza perdere accuratezza; l'adozione di un classificatore lineare pesato per il sentiment, visto il margine emerso nei test (F1 0,708); e una valutazione dell'LLM su un campione più ampio, possibilmente con qualche esempio nel prompt (few-shot), per capire se il vantaggio osservato regge.